المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⵓⵙⵙⵏⵉⵡⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

COMPTE RENDU DU TP-1

1ÈRE ANNÉE – FILIÈRE INTELLIGENCE ARTIFICIELLE ET CYBERSÉCURITÉ

MODULE : ARCHITECTURE LOGICIELLE

Réalisé par :

ASIM ZINEB

Encadré par :

Pr. BE ELBAGHAZAOUI

Année unniversitaire:2025_2026

Objectifs

Ce TP consiste à créer une application Spring Boot monolithique avec une API REST CRUD complète pour la gestion de produits. L'architecture suit le pattern en couches : Entity,
Repository, Service, Controller.

Outils utilisés

- JDK 17, Maven, IntelliJ IDEA.
- Spring Boot.
- PostgreSQL.
- Postman.

Réalisation du TP

1. Initialisation du Projet

Le projet a été généré via Spring Initializr avec les dépendances suivantes :

- Spring Web : Création des endpoints REST.
- Spring Data JPA : permet d'interagir avec la base de données.
- PostgreSQL Driver : pour la connexion au SGBD PostgreSQL
- Lombok : Génération automatique du code répétitif.
- Validation : la validation des données en entrée.

2. Configuration

La base de données PostgreSQL "ecommerce" a été créée localement. Le fichier application.properties a été configuré comme suit :

```
spring.application.name=monolith
spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.datasource.driver-class-name=org.postgresql.Driver
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

3. Implémentation

- **Entité Product**

L'entité Product est le centre du modèle de données. Elle correspond à la table products en base de données grâce aux annotations JPA. Chaque produit possède un identifiant généré automatiquement, un nom obligatoire, une description facultative, un prix positif et un stock non négatif. Les annotations de validation garantissent la cohérence des données avant leur enregistrement. L'utilisation de Lombok permet de réduire le code répétitif et de rendre la classe plus claire et facile à maintenir.

```java
package com.ecommerce.monolith.product.model;
import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.math.BigDecimal;
@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;
    @Column(length = 1000)
    private String description;
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;
    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
}
```

- **Repository**

Le ProductRepository étend JpaRepository de Spring Data JPA, ce qui permet d'avoir automatiquement les opérations CRUD sans écrire de SQL. Il prend deux paramètres : Product (l'entité) et Long (le type de la clé primaire).
Une méthode findByNameContainingIgnoreCase a été ajoutée pour rechercher des produits par nom sans tenir compte des majuscules/minuscules, et Spring génère la requête automatiquement.

```java
package com.ecommerce.monolith.product.repository;
import com.ecommerce.monolith.product.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByNameContainingIgnoreCase(String name);
}
```

- **Le Service**

La classe ProductService gère la logique métier de l'application. Elle est annotée @Service pour être détectée par Spring et @Transactional pour exécuter les opérations dans une transaction.
L'injection des dépendances se fait via le constructeur avec @RequiredArgsConstructor de Lombok.
Les méthodes de lecture utilisent @Transactional(readOnly = true) pour de meilleures performances.
La méthode getById lance une exception si le produit n'existe pas, que le controller pourra transformer en réponse HTTP adaptée.

```java
package com.ecommerce.monolith.product.service;
import java.util.List;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
@Service
@RequiredArgsConstructor
@Transactional
public class ProductService {
    private final ProductRepository repository;
    @Transactional(readOnly = true)
    public List<Product> getAll() { return repository.findAll(); }
    @Transactional(readOnly = true)
    public Product getById(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found with id: " + id));
    }
    public Product create(Product product) { return repository.save(product); }
    public Product update(Long id, Product details) {
        Product product = getById(id);
        product.setName(details.getName());
        product.setDescription(details.getDescription());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());
        return product;
    }
    public void delete(Long id) {
        Product product = getById(id);
        repository.delete(product);
    }
}
```

- **Le Controller**

Le ProductController gère la présentation de l'application. Annoté @RestController, il expose les endpoints REST sous /api/products.
Chaque méthode HTTP correspond à un CRUD : GET pour lire, POST pour créer, PUT pour mettre à jour et DELETE pour supprimer.
@Valid vérifie automatiquement les données entrantes selon les contraintes de l'entité.
Chaque méthode renvoie un ResponseEntity avec le code HTTP adapté : 200 pour lecture/mise à jour, 201 pour création, 204 pour suppression.

```java
package com.ecommerce.monolith.product.controller;
import java.util.List;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.service.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
@RestController  no usages
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {
    private final ProductService service;  5 usages
    @GetMapping  no usages
    public ResponseEntity<List<Product>> getAll() { return ResponseEntity.ok(service.getAll()); }

    @GetMapping("/{id}")  no usages
    public ResponseEntity<Product> getById(@PathVariable Long id) { return ResponseEntity.ok(service.getById(i

    @PostMapping  no usages
    public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
        Product created = service.create(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")  no usages
    public ResponseEntity<Product> update(@PathVariable Long id, @Valid @RequestBody Product product) {
        return ResponseEntity.ok(service.update(id, product));
    }

    @DeleteMapping("/{id}")  no usages
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

4. Test avec Postman

L'application a été lancée avec succès en utilisant la commande mvn spring-boot:run, puis on teste les méthodes CRUD pour vérifier leur fonctionnement :

- ✓ **Liste tous les produits**

Méthode : GET
URL : http://localhost:8080/api/products

La réponse

Body Cookies Headers (5) Test Results Status: 200 OK Time: 18

Pretty Raw Preview Visualize JSON

```
[
    {
        "id": 1,
        "name": "Smartphone Samsung",
        "description": "Téléphone 256Go",
        "price": 899.99,
        "stock": 50
    }
]
```

- ✓ **Créer un produit**

Méthode : POST
URL : http://localhost:8080/api/products
Header : Content-Type : application/json

Corps de la requête :

POST http://localhost:8080/api/products

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
    "name": "Laptop Dell XPS",
    "description": "Ordinateur portable haute performance",
    "price": 1299.99,
    "stock": 10
}
```

La réponse :

Body Cookies Headers (5) Test Results Status: 201 Created Tim

Pretty Raw Preview Visualize JSON

```
{
    "id": 2,
    "name": "Laptop Dell XPS",
    "description": "Ordinateur portable haute performance",
    "price": 1299.99,
    "stock": 10
}
```

- ✓ **Modifier un produit**

Méthode : PUT
URL: http://localhost:8080/api/products/2
Header : Content-Type : application/json

Corps de la requête :

PUT http://localhost:8080/api/products/2

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
{
    "name": "Laptop Dell XPS 15",
    "description": "Nouvelle version améliorée",
    "price": 1499.99,
    "stock": 8
}
```

La réponse :

Body Cookies Headers (5) Test Results Status: 200 OK

Pretty Raw Preview Visualize JSON

```
{
    "id": 2,
    "name": "Laptop Dell XPS 15",
    "description": "Nouvelle version améliorée",
    "price": 1499.99,
    "stock": 8
}
```

- ✓ **Supprimer un produit**

Méthode : DELETE
URL: http://localhost:8080/api/products/2

DELETE http://localhost:8080/api/products/2

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary

This request does not have a body

Body Cookies Headers (3) Test Results Status: 204 No Content Time:

Pretty Raw Preview Visualize Text

Exercice : Entité Category et relation ManyToOne

1. Création de l'entité Category

L'entité Category a été ajoutée afin de regrouper les produits. Cette entité contient un identifiant et un nom. Elle permet d'organiser les produits de manière logique en les associant à des catégories spécifiques.

```
package com.ecommerce.monolith.category.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.NotBlank;
import lombok.*;

@Entity  4 usages
@Table(name = "categories")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Category {

    @Id  no usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Category name is required")  no usages
    @Column(nullable = false)
    private String name;
}
```

2. Mise en place de la relation entre Product et Category

Une relation de type ManyToOne a été établie entre les entités Product et Category. Cela signifie qu'un produit appartient à une seule catégorie, tandis qu'une catégorie peut contenir plusieurs produits. Cette relation est matérialisée par une clé étrangère dans la table des produits.

3. Création des Repository

```
package com.ecommerce.monolith.category.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.rest.core.annotation.RepositoryRestResource;

import com.ecommerce.monolith.category.model.Category;

@RepositoryRestResource(path = "categories")  2 usages
public interface CategoryRepository extends JpaRepository<Category, Long> {
}
```

4. Implémentation du Service

```java
package com.ecommerce.monolith.category.service;
import ...
@Service  1 usage
@RequiredArgsConstructor
@Transactional
public class CategoryService {
    private final CategoryRepository repository;  4 usages
    @Transactional(readOnly = true)  no usages
    public List<Category> getAll() { return repository.findAll(); }
    @Transactional(readOnly = true)  2 usages
    public Category getById(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Category not found with id: " + id));
    }
    public Category create(Category category) { return repository.save(category); }
    public Category update(Long id, Category details) {  no usages
        Category category = getById(id);
        category.setName(details.getName());
        return category;
    }
    public void delete(Long id) {  no usages
        Category category = getById(id);
        repository.delete(category);
    }
}
```

5. Création du Controller

```java
package com.ecommerce.monolith.category.controller;
import ...
@RestController  no usages
@RequestMapping("/api/categories")
@RequiredArgsConstructor
@Validated
public class CategoryController {
    private final CategoryService service;  5 usages
    @GetMapping  no usages
    public ResponseEntity<List<Category>> getAll() { return ResponseEntity.ok(service.getAll()); }

    @GetMapping("/{id}")  no usages
    public ResponseEntity<Category> getById(@PathVariable Long id) { return ResponseEntity.ok(service.getById(

    @PostMapping  no usages
    public ResponseEntity<Category> create(@Valid @RequestBody Category category) {
        Category created = service.create(category);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")  no usages
    public ResponseEntity<Category> update(@PathVariable Long id, @Valid @RequestBody Category category) {
        return ResponseEntity.ok(service.update(id, category));
    }

    @DeleteMapping("/{id}")  no usages
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
```

6. Tests avec Postman

✓ Créer une catégorie

POST http://localhost:8080/categories

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Informatique"
3 }
```

Body Cookies Headers (9) Test Results Status: 201 Created Time: 464

Pretty Raw Preview Visualize JSON

```
1 {
2     "name": "Informatique",
3     "_links": {
4         "self": {
5             "href": "http://localhost:8080/categories/10"
6         },
7         "category": {
8             "href": "http://localhost:8080/categories/10"
9         }
10    }
11 }
```

✓ **Créer un produit avec catégorie**

POST http://localhost:8080/categories

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Laptop",
3   "description": "PC portable",
4   "price": 8000,
5   "stock": 10,
6   "category": "http://localhost:8080/categories/1"
7 }
```

Body Cookies Headers (9) Test Results Status: 201 Created

Pretty Raw Preview Visualize JSON

```
1 {
2     "name": "Laptop",
3     "_links": {
4         "self": {
5             "href": "http://localhost:8080/categories/13"
6         },
7         "category": {
8             "href": "http://localhost:8080/categories/13"
9         }
10    }
11 }
```

Conclusion

Ce premier TP a permis d’expérimenter la mise en place d’une application backend structurée autour d’une architecture monolithique. La réalisation d’une API complète a montré l’importance de la séparation des responsabilités entre les différentes couches et la cohérence du modèle de données. L’introduction d’une relation entre entités a illustré la richesse des interactions possibles dans une base et renforcé la compréhension de la persistance. Enfin, les tests effectués ont validé la fiabilité de l’ensemble et confirmé la pertinence de l’approche adoptée.